

Technische Hochschule Ingolstadt
Fakultät Informatik

Augmented and Virtual Reality Applications Practical Exercises

02: Assignment 01 - Roll a Ball

Introduction to AR/VR

Overview

- Submission and upload rules
- Task 1: Create The Scene
- Task 2: Player Movement
- Task 3: Camera Movement
- Task 4: Add Walls
- Task 5: Create Collectibles
- Task 6: Collision Detection
- Task 7: Bonus Tasks

Practical Exercises

Introduction to Unity Part 2 – Assignment 1

Submission Guidelines

- One team member uploads the assignment; points are automatically assigned to all team members.
- The assignment consists of two parts:
  - The minimal version: to be completed and uploaded by everybody. Mandatory! (Pages 5 - 35)
  - Additional bonus tasks: optional (Page 36-37)
- Either way, upload **one** ZIP folder (max. 400 MB) containing:
  - one **MP4 video**: Show the minimal version. (Free recording tool: OBS Studio)
  - another **MP4 video:** implemented bonus tasks.
  - **TXT, MD, or PDF file**: List your implemented bonus tasks (e.g., “Completed bonus tasks: a, d, f”) or any other additional notes.
- File naming convention:
  surname1_surname2.zip
  Consisting of:
  surname1_surname2_minimal.mp4
  surname1_surname2_bonus.mp4 (if implemented)
  surname1_surname2.txt

  Important: Incorrect ZIP file naming may result in points not being awarded!
- **Deadline: Thursday, October 29, 23:59**

Practical Exercises

Introduction to Unity Part 2 – Assignment 1

Assignment 1: Roll A Ball Game (Introduction to Unity, Part 2)

In the second part of our Unity introduction, we will create the small 3D game “Roll-a-Ball” from *Unity Learn*.
Ours is a slightly adjusted version which replaces some code from the original online tutorial. Other details are changed/omitted due to time constraints.

Goal: Bring GameObjects to life using C# scripts. No C# experience needed. We will work through everything step by step.
(If you know Java, lucky you.)

Task 1: Create the scene

Practical Exercises

Introduction to Unity Part 2 – Assignment 1

Task 1: Create the scene

a) Start the exercise by creating a new project (Template: “Universal 3D”) called “RollABall” .
Project Pane Window → Assets → Scenes: rename “SampleScene” to “rollaball”.

b) Create a new GameObject → 3D Object → Plane and rename it to **Ground** in the Inspector. Adjust the Transform component:
Position: (0, 0, 0)
Rotation: (0, 0, 0)
Scale: (2, 1, 2)

c) Let’s create our “Player”, which will be a rolling sphere:
Create a new GameObject → 3D Object → Sphere, rename to **Player**. Transforms:
Position: (0, 2, 0)
Rotation: (0, 0, 0)
Scale: (1, 1, 1)

Practical Exercises

Introduction to Unity Part 2 – Assignment 1

Task 1: Create the scene

d) The gray sphere is hard to see on the gray ground. In the Project View, create a new folder called “Materials”.
In it, create a new Material, rename it “Background”. For this material, set the Base (Map) Color as you like.
Assign it to the Ground plane.

e) Directional Light → Transform component: Y-Rotation to 60 (to achieve a different illumination of the scene).
Try to understand why only changing the rotation affects the lighting of the scene, but changing the lights’ position does not.

Task 2: Player Movement

Task 2: Player Movement

a) Add physics: Select **Player** and add a **Rigidbody** component.

b) Create and attach a script:
In the Project window, create a folder named Scripts. Select **Player** → Inspector → Add Component → New Script.
Name the script **PlayerController** and press Enter (or click “Create and Add”).
Move the script into the Scripts folder to keep the project organized.

c) Open the script for editing:
Open **PlayerController** in your development environment (e.g. Visual Studio Code) using any of these methods:

- Double-click the script in the Project window (easiest).
- Select the script and click “Open” in the Inspector.
- Right-click the script and select Open.
- In the Player Inspector, open the script component’s ⋮ menu and select “Edit Script”.

Task 2: Player Movement

d) In the script editor, you will see the automatically created body of the script. The gray colored comments (starting with //) already indicate where in the life cycle of the game the individual methods are executed.

```
1   using UnityEngine;
2
3   public class PlayerController : MonoBehaviour
4
5   {
6       // Start is called once before the first execution of Update after the MonoBehaviour is created
7       void Start()
8       {
9
10      }
11
12      // Update is called once per frame
13      void Update()
14      {
15
16      }
17  }
18
```

Practical Exercises

Introduction to Unity Part 2 – Assignment 1

Task 2: Player Movement

Add line 2 (enables reading Keyboard Input):

```
1   using UnityEngine;
2   using UnityEngine.InputSystem;
3
4   public class PlayerController : MonoBehaviour
5   {
```

Practical Exercises

Introduction to Unity Part 2 – Assignment 1

Task 2: Player Movement

d) Understand the game lifecycle:

`Start()`: Runs once when the game starts; used for initialization.
`Update()`: Runs every frame, before it is rendered.
`FixedUpdate()`: Runs at fixed time intervals, making it better suited for physics calculations.

e) Replace `Update()` with `FixedUpdate()` for physics-based movement.

We'll use Unity's Input System inside `FixedUpdate()` to check our keyboard input continuously during GamePlay:

Left / Right arrows: Move sideways (along the X-axis).
Up / Down arrows: Move forward/backward (along the Z-axis).

Practical Exercises

Introduction to Unity Part 2 – Assignment 1

Task 2: Player Movement

```
private void FixedUpdate()
{
    float moveX = 0f;
    float moveZ = 0f;

    // Read keyboard input using the modern Input System
    if (Keyboard.current != null)
    {
        // Left/right movement
        if (Keyboard.current.leftArrowKey.isPressed)
            moveX = 1f;
        else if (Keyboard.current.rightArrowKey.isPressed)
            moveX = -1f;

        // Forward/backward movement
        if (Keyboard.current.upArrowKey.isPressed)
            moveZ = -1f;
        else if (Keyboard.current.downArrowKey.isPressed)
            moveZ = 1f;
    }
```

We initialize two float variables (`0f` represents the **f**loat value 0.0) called `move` (twice in X and Z) to store some numerical value for horizontal input (+1: left, -1: right, 0: no input) and “vertical” respectively.

← You can swap +1 and -1 to reverse directions.

← Remember: up/down keys = forward/backward.
(We discussed 3D/2D navigation in the lecture.)

Task 2: Player Movement

f) To apply the force, we need access to the **Rigidbody** component of **Player**.
To do this, we first define a private attribute `rb` of type `Rigidbody:`

```
1  using UnityEngine;
2  using UnityEngine.InputSystem;
3
4  public class PlayerController : MonoBehaviour
5  {
6      private Rigidbody rb;
7
8      private void Start()
9      {
```

Next we assign this `rb` attribute inside the `void Start()` block:

```
rb = GetComponent<Rigidbody>();
```

This line searches for a “Rigidbody” component on the GameObject the script is attached to (ours is attached to **Player**), and assigns it to the `rb` variable so we can use it later.

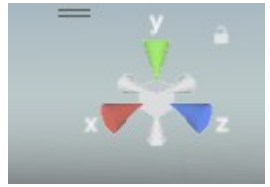
(Make sure the block is closed (the curly brace **}** below the line).

Task 2: Player Movement

g) Save and refresh:
Save your script (Ctrl + S). If changes don't appear, use Menu → Assets → Refresh (Ctrl + R). If needed, try Reimport or Reimport All (may take a minute).

Press the Play button (above Scene View) to test. The Player falls due to gravity (it is a RigidBody: a "physical object" with mass and collision detection), but the arrow keys have no effect yet. To move the Player, we must apply a force based on keyboard input.

h) Create the movement vector:
A Vector3 represents a 3D vector with X, Y, and Z components.



General 3D / linear algebra knowledge:
a vector in 3D consists of 3 floats ("vector three" in short) describing the 3 coordinate axes X, Y, Z.
A vector has a direction and a length. A force is a vector. (Here: gravity or acceleration).

Inside the `FixedUpdate()` block, declare a `Vector3` variable named `movement` and assign it (moveX, 0f, moveZ):
Write the line of code yourself before looking at the next slide.

Practical Exercises

Introduction to Unity Part 2 – Assignment 1

Task 2: Player Movement

```
44          else if (Keyboard.current.downArrowKey.isPressed)
45              moveZ = 1f;
46      }
47
48      // Create the movement vector
49      Vector3 movement = new Vector3(moveX, 0f, moveZ);
50
```

h) In `FixedUpdate()`, we now want to apply this force vector to `rb`. Find out how to do this by looking into the Unity Scripting Reference

Write the function yourself (below the line in above image) using the `movement` vector and `ForceMode.Force`.
Don't forget the semicolon ;

Save and test: Press Play and use the arrow keys to move the ball. It barely moves. If at all.

Practical Exercises

Introduction to Unity Part 2 – Assignment 1

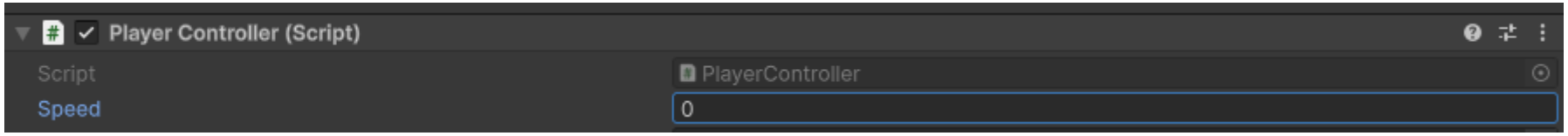
Task 2: Player Movement

i) We want to be able to adjust the speed. Trick in Unity:
Public variables in a script are turned into editable fields in the Inspector. This allows for adjusting values without changing the code.
Above the `void Start()` block, declare:

```
public float speed;
```

Within `FixedUpdate()`, in the `rb.AddForce()` function you wrote, multiply the `movement` vector by `speed`.

Save the script. A “Speed” input field will appear in the Player’s Inspector:



Experiment: Adjust the Speed value in Play mode until the ball moves as expected.
Note: Changes made in Play mode are reset when you stop the game.

Task 3: Camera Movement

Task 3: Camera Movement

a) Make the camera follow the Player:
The camera currently stays still, so the ball eventually moves out of view. Let`s make it follow the ball.
First, reset **Main Camera**'s Transforms, then set:

Position: (0, 10, 10)
Rotation: (45, 180, 0)

Making the camera a child of Player in the Hierarchy would make it follow the ball as well - but also rotate with it.
Instead, we'll use a few lines of code to make the camera follow the Player's position only, without copying its rotation.

Task 3: Camera Movement

b) Create the CameraController script:
Select **Main Camera** → Inspector → Add Component → New Script. Name it **CameraController** and move it into the Scripts folder.

Open the (empty) script. Above `Start()`, declare:

```
public GameObject player;
private Vector3 offset;
```

Save the script. In the Main Camera Inspector, drag **Player** from the Hierarchy into the new “Player” input field. If it doesn’t appear: Menu → Assets → Refresh.

Task 3: Camera Movement

c) First, we calculate the inital camera offset from the player:
Inside `void Start()`, add:

```
offset = transform.position - player.transform.position;
```

This calculates the initial distance between the **Main Camera** and the **Player** and stores it in the `offset` vector variable.
Important to understand:
`transform.position` refers to the camera position: since the script is attached to **Main Camera**, `transform` automatically refers to its “Transform” component. No need to reference **Main Camera** explicitly.
`player.transform.position` refers to the **Player**’s position. Here, we need to reference `player`.

Important to know as well: every Unity GameObject has (at least) a Transform component. Even an empty GameObject.

Task 3: Camera Movement

d) Add initial camera offset to **Player** position during GamePlay:
Replace `Update()` with `LateUpdate()`, which runs each frame, but **after** the game logic.
Inside `LateUpdate()`, set the camera position to the Player's position:

```
transform.position = player.transform.position;
```

Save, refresh, and test in Play mode. The camera follows the Player, but we want to keep the Sphere in view.
Add the offset vector to the Player's position to maintain the camera's distance and viewing angle:

```
transform.position = player.transform.position + offset;
```

Save, refresh, and test in Play mode.
Your task: Review steps b), c) and d) to understand how the camera follows the Player *while maintaining its offset*.

Task 4: Add walls

Practical Exercises

Introduction to Unity Part 2 – Assignment 1

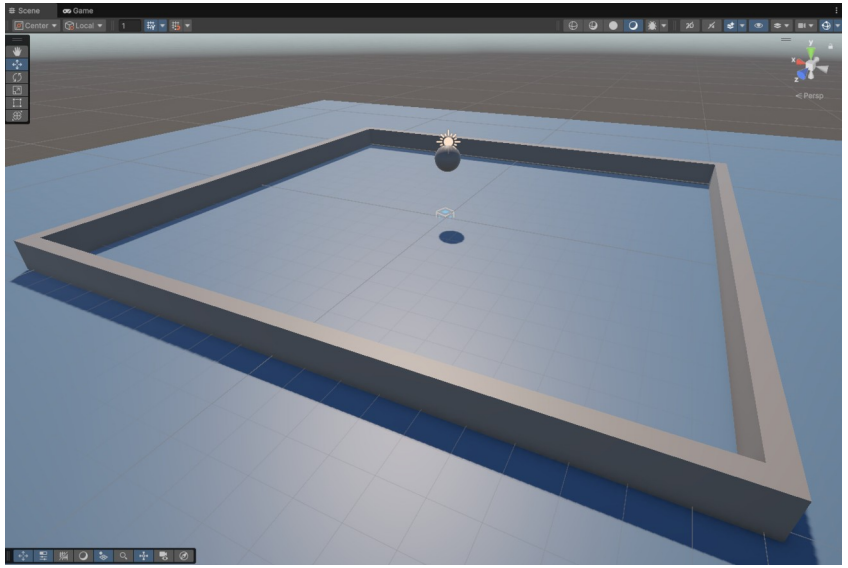
Task 4: Add walls

a) Create the boundary walls:
Create an empty GameObject named **Walls** at position (0, 0, 0). Add a Cube named **West Wall** as its child.
Set its Transform:
Position: (-10, 0, 0)
Rotation: (0, 0, 0)
Scale: (0.5, 2, 20.5)

Duplicate the wall (Ctrl + D), rename it **East Wall**, and set Position X to 10.

b) Create the north and south walls using the same approach.

Test: Save, enter Play mode, and check that the ball bounces off the walls.



Task 5: Create “Collectibles”



Task 5: Create Collectibles

a) We want to create some collectibles for our Player. Create a Pick Up object:
Add a Cube named **Pick Up**. Set its Transform:
Position: (0, 0.5, 0)
Rotation: (45, 45, 45)
Scale: (0.5, 0.5, 0.5)
(Link the Scale values to change all three axes simultaneously).

b) Make the static collectible rotate:
Add a script named **Rotator** to **Pick Up**.
In `Update()`, rotate the object using `transform.Rotate()` and multiply the rotation vector by `Time.deltaTime` to ensure frame-rate-independent rotation.

```
11      // Update is called once per frame
12      void Update()
13      {
14          transform.Rotate(new Vector3(15, 30, 45) * Time.deltaTime);
15      }
16  }
17
```

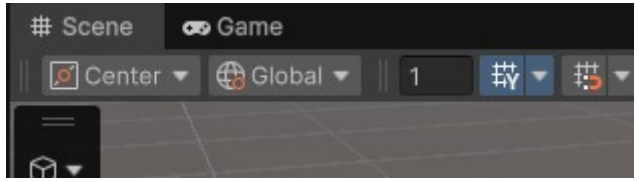
Test: Save, refresh, and enter Play mode. The collectible rotates, but the Player bounces off it instead of collecting it. Let's fix that.

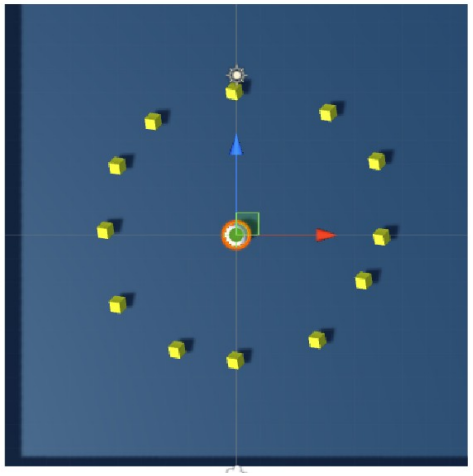
Task 5: Create Collectibles

c) Create a prefab:
Create a **Prefabs** folder in the Project window. Drag **Pick Up** from the Hierarchy into the folder to create a prefab (name turns blue). Remember from first practical: changes to the prefab will be applied to all its instances.

d) Duplicate and arrange collectibles:
Create an empty GameObject named **Pick Ups** and reset its Transform. Make **Pick Up** its child and duplicate it 10–12 times (Ctrl + D).

Manually distribute the copies by moving them around in the Scene View:
Select a **Pick Up** object to display its Transform handles. Switch from Local to Global coordinates (shortcut X) to align the handles with the world axes and position collectibles easily along the ground.





Task 5: Create Collectibles

e) Change the Collectible's Appearance:
In the Project window, create a Material named **PickUpMaterial** and move it into the Materials folder.
Change the material's color.

Select the **Pick Up** Prefab in the Project window.
In the Inspector → Mesh Renderer → Materials → Element 0, click the small circle and select **PickUpMaterial**.



Task 6: Collision Detection

Task 6: Collision Detection

a) To collect collectibles, collisions between them and **Player** must be detected. “Collider” components are responsible for this. These then trigger an event. To the **PlayerController** script, add the following method:

```
public class PlayerController : MonoBehaviour
{
    private Rigidbody rb;
    public float speed = 10f;

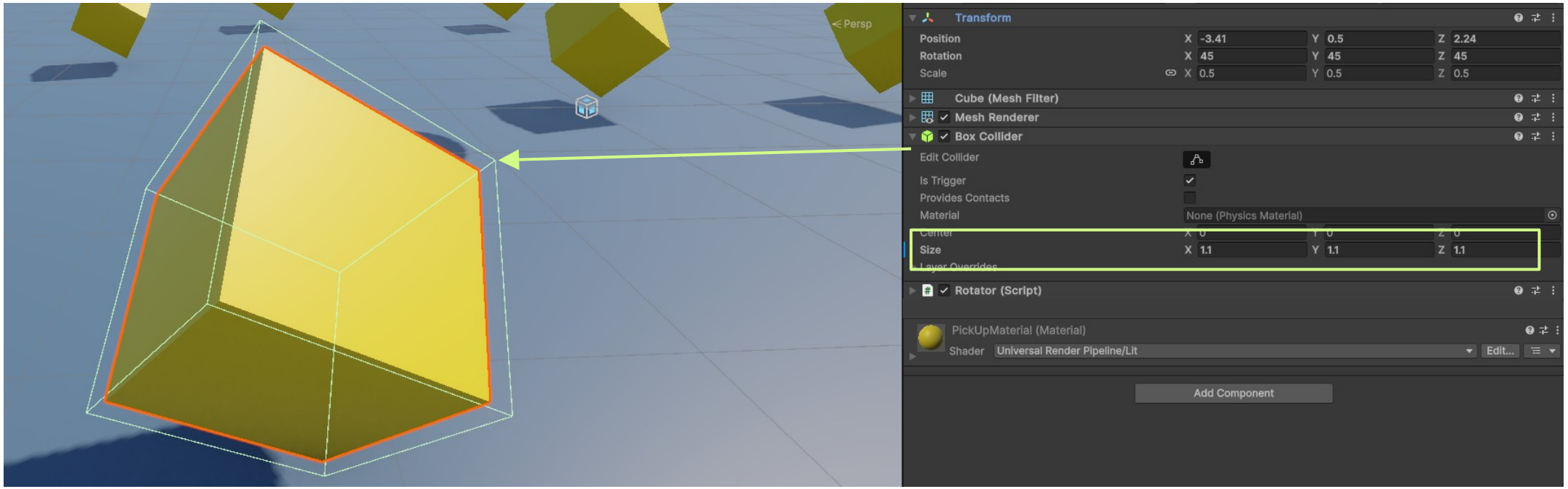
    private void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag("PickUp"))
        {
            other.gameObject.SetActive(false);
        }
    }

    void Start()
```

Once you *enter* the *Trigger* zone of some other object, do something….

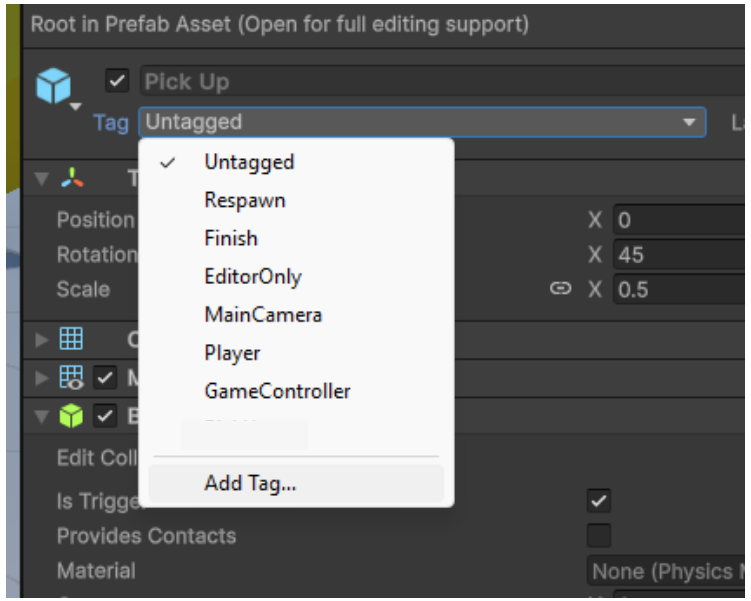
Task 6: Collision Detection

This trigger event is actually a collider zone and was auto-created when we created the collectibles as a Cube GameObject. Here I increased its size to make it visible (you don't need to do this - only for understanding purposes).



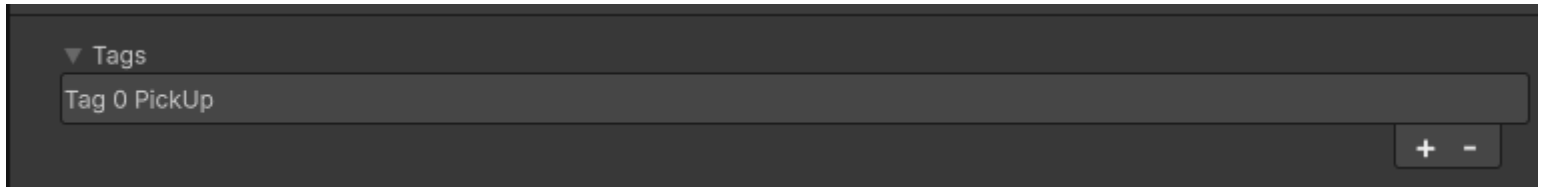
Task 6: Collision Detection

Whenever **Player** collides with an object (“other object”), we check whether it is a **Pick Up**. If so, we’ll set it to inactive to make it disappear.
b) How do we recognize a **Pick Up**? We provide the Prefab with a so-called **tag:** Select the **Pick Up** Prefab → Inspector Pane → Tag → Add Tag...

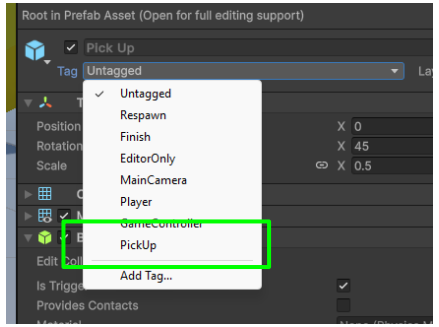


Task 6: Collision Detection

Click the “plus” symbol, call the new tag **PickUp** - if you want a different name, you’d need to change the code accordingly. Hit enter:



In the script, `CompareTag()` checks whether the collided object has this tag.
Important: Creating a tag does not assign it. Again select the **Pick Up** Prefab → Inspector → Tag → PickUp.



Check a few instances in the Hierarchy to confirm the tag was applied to all copies:



Task 6: Collision Detection

c) Select the **Pick Up** Prefab → Box Collider → Is Trigger.
Save, refresh, and test in Play mode. Collectibles should disappear when the Player touches them.

Congratulations! You've completed the minimal version of the assignment. Bonus tasks follow on the next slides.

Task 7: Bonus Tasks

Task 7: Bonus Tasks

Choose which tasks to implement. Read all tasks first! Maximum: 2.5 points (Capped. Additional points do not count). Show all implemented features clearly in your video and list them in the submission text file.

7a) Play a sound when the Player collects a Pick Up. **0.3 Points**

7b) Add at least 8 (thin) walls to create a maze. Expand the play area if needed. **0.3 Points**

7c) Enhance the level design with textures, maps, and appealing materials (e.g. wooden ground, textured walls, different Player, Pick Ups). **0.4 Points** (0.1 each: Ground, Walls, Player, Pick Ups)

7d) Level design: Create a nicely lit environment: Add at least 3 additional lights of different types, colors, intensities, or temperatures. **0.2 Points**

7e) Add soft movement to the existing camera follow system. **0.1 Points**

Practical Exercises

Introduction to Unity Part 2 – Assignment 1

7f) Display the score during GamePlay and increase it for each collected item. **0.3 Points**

7g) Add a countdown timer and display “Time’s up!” when time runs out. **0.4 Points**

7h) Enrich the environment: Add at least 5 different, freely available, thematically coherent 3D assets (e.g., objects, characters, plants, furniture...). The Player should bounce off them on collision. **0.5 Points** (Warning: quite similar to 7k. Read first!)

7i) Display “Congratulations!” when all Pick Ups are collected. **0.3 Points**

7j) Add fireworks or a confetti particle system when all Pick Ups are collected. **0.7 Points**

7k) Re-theme the whole game (colors, music, assets, environment, lights) for a **cohesive look and scenery**.
Choose a theme (e.g., underwater, desert, space). Decide color palette, choose according assets, and mood. **1.3 Points**

End.